Receipt Chains:

Cryptographic Conservation of Consequence

The Chatman Equation Thesis Program
(Praxis / Capability Physics)

Genesis — Receipt Cryptography, Paper 02

Abstract

The keystone of this series established that trust among bounded agents is manufactured by a *faithful projection* of an unbounded execution onto a coordinate small enough to fit a verifier's bounded context, cryptographically bound so the projection cannot lie about the interior it summarizes [1]. This paper supplies the cryptography that makes the binding real. We define the *frame* — the fixed-width, 128-byte, cache-line-aligned `OcelCausalFrame` that encodes a single manufacturing step and commits the full 256-bit payload digest into its object-reference slots — and the *chain rule* $h_+ = \mathsf{H}(h_- \parallel \beta(\mathsf{fr}))$, a rolling BLAKE3 commitment that folds each frame onto its causal predecessor. We prove, from an explicit collision-resistance axiom, the **Faithful Chain Theorem**: perturbing any committed field forces a hash collision, so a bounded verifier who recomputes one link rejects the perturbation reading $O(1)$ per frame and $O(\kappa)$ at the boundary. We prove the **Conservation of Consequence Theorem** in its receipt-cryptographic form: every actuated artifact occupies *exactly one* chain position, the map from actuation to chain value is injective up to collision, and each artifact carries *at least one* committed admitted cause. We give the **Tamper Localization Theorem** for the staged validator (`schema`, `chain_recompute`, `chain_linkage`, `monotonic`, `token_replay`), which does not merely reject a corrupted ledger but names the first divergent frame and the stage that caught it. We formalize Ed25519 signing as *self-contained authenticity*, separating integrity (what the chain alone proves) from authorship (what a signature adds), and describe the content-addressed *receipt lake* with its OCEL 2.0 / PROV-O mapping and its index-scale query direction. We close with the **Integrity–Virtue Scope Theorem**: a proof of what a receipt chain establishes and, equally rigorously, what it does not — the guardrail that keeps a cryptographic apparatus from being mistaken for a warrant of goodness.

Contents

Chapter 1

Introduction: The Receipt Is a Commitment, Not a Copy

The keystone thesis derives the standard of trust available to a civilization of agents who cannot comprehend one another: not comprehension, but a bounded, faithful, replayable receipt [1]. That derivation leaves one thing on credit. It *assumes* a collision-resistant hash H and asserts that a receipt "cannot lie about the interior it summarizes." This paper pays the debt. We construct the object that discharges the assumption — the receipt chain — and prove, rather than assert, exactly which lies it forecloses and which it does not.

A receipt is often imagined as a *copy*: a smaller transcript of what happened, trusted because it looks like the original. That picture is wrong at every scale that matters. A copy is faithful only if the reader compares it to the original, which is precisely the comprehension cost the keystone showed we cannot pay. The receipt in this doctrine is not a copy but a *commitment*: a short value h_+ such that no adversary can exhibit a different history yielding the same h_+ without breaking a cryptographic assumption. The verifier never compares the receipt to the interior. It recomputes one arithmetic step and checks equality. Fidelity is enforced by the algebra of the hash, not by the diligence of the reader.

The three questions. A receipt discipline must answer three distinct questions, and the doctrine's discipline is to never conflate them:

(Q1) **Integrity.** Was this history altered after the fact? (Answered by the chain.)

(Q2) **Authenticity.** Who vouches for this history? (Answered by a signature.)

(Q3) **Virtue.** Was the history *good* — were the right obligations checked, is the artifact fit for the world? (*Not* answered by cryptography at all.)

Chapters 4–6 answer (Q1); Chapter 7 answers (Q2); Chapter 9 proves the boundary of (Q3). Keeping these separate is not pedantry. A system that answers (Q1) and pretends it has answered (Q3) is exactly the overclaim the keystone's antibody clause forbids [1].

Grounding. Every construction here is realized in the praxis codebase, and we name the site so a reader can trace math to machine. The frame is `bcinr_powl_receipt::causal_receipt::OcelCausalFrame`; its serialization is `to_hash_bytes`; the chain rule is `OcelCausalReceipt::chain`; the single

frame-construction site shared by emission and replay is `praxis_core::law::build_admission_frame` with `chain_from_frame`; the persisted record is `praxis_core::receipt_record::ReceiptRecord`; the staged validator is `praxis_core::receipt_validator::ReceiptValidator`; signing is `praxis_core::signing`. Where a size, a field order, or a constant appears below, it is the size, field order, or constant the code compiles with, and in several cases the code asserts it at compile time.

Notation. We reuse the series' notation: $\mathcal{O}$ raw observations, $\mathcal{O}^*$ the admitted sub-language, α the admission retraction with refusal $\perp$, $\mu : \mathcal{O}^* \to \mathcal{A}$ the manufacturing morphism, Σ the space of full execution traces, $\mathcal{R}$ the receipt space, H the collision-resistant hash, $\mathrm{dg}(\cdot) = \mathsf{H}(\cdot)$ a digest, and h_-, h_+ the chain values before and after a step. The keystone's law-bearing frame is here made concrete as the byte-exact `OcelCausalFrame`.

Chapter 2

The Frame: A Fixed-Width Encoding of One Manufacturing Step

2.1 Why fixed width

A commitment scheme is only as honest as its encoding. If two distinct histories can serialize to the same bytes, the hash of those bytes commits to neither unambiguously. The first requirement on a frame is therefore *unambiguous, length-determined serialization*: the reader of the byte body must be able to parse each field without a length prefix, because the layout is fixed. The praxis frame achieves this by being a `repr(C, align(64))` struct of a single compile-time-constant size.

Definition 2.1 (Frame)**.** A *frame* fr is the record

$$\mathsf{fr} = \big\langle \texttt{instruction\_id}, \texttt{fired\_mask}, \texttt{denial}, \texttt{obj\_refs}[0{:}8], \texttt{ts\_ns}, \texttt{activity\_idx}, \texttt{node\_kind}, \texttt{prior\_hash} \big\rangle,$$

laid out in memory as a `repr(C, align(64))` structure occupying exactly 128 bytes — two 64-byte cache lines — with the field widths of Table 2.1. The in-memory layout carries 5 bytes of interior padding (`pad`) so that the 256-bit `prior_hash` lands aligned; the padding is structural, not semantic.

Remark. The 128-byte size is not a description but an invariant the compiler enforces. The crate contains the const assertion

```
const _:  () = { assert!(size_of::<OcelCausalFrame>() == 128) };
```

so a change to any field width that broke the two-cache-line layout would fail to compile. Fixed width is thus a mechanized property (in the sense of the foundations paper's mechanized-totality results), not a convention a maintainer must remember.

2.2 The hash body: what the frame commits

The frame's in-memory image is 128 bytes, but the bytes fed to the hash are a distinct, smaller, canonical serialization: the padding is excluded, and every integer is written little-endian in a fixed order. This is the object the chain actually commits to.

Byte range	Field	Width	Meaning
$[0, 8)$	`instruction_id`	u64 LE	monotone step identity within a run
$[8, 16)$	`fired_mask`	u64 LE	scatter of active denial lanes
$[16, 24)$	`denial.0`	u64 LE	denial-polarity word (admission verdict)
$[24, 56)$	`obj_refs`	8× u32 LE	*payload digest* (§2.3)
$[56, 64)$	`ts_ns`	u64 LE	wall-clock nanoseconds
$[64, 66)$	`activity_idx`	u16 LE	POWL activity-table index
$[66, 67)$	`node_kind`	u8	POWL node classifier (XOR/SEQ/LOOP/...)
$[67, 99)$	`prior_hash`	32 B	BLAKE3 of the causal predecessor

Table 2.1: The 99-byte canonical hash body $\beta(\mathsf{fr})$ produced by `to_hash_bytes`. The 5 interior `pad` bytes of the 128-byte in-memory frame are *not* hashed. Total committed width: $8 + 8 + 8 + 32 + 8 + 2 + 1 + 32 = 99$ bytes.

Definition 2.2 (Hash body)**.** The *hash body* $\beta(\mathsf{fr}) \in \{0,1\}^{8\cdot 99}$ is the 99-byte serialization produced by `to_hash_bytes`, with the field layout of Table 2.1. It is a total, injective function of the semantic fields of fr (all fields except the structural `pad`): distinct field tuples yield distinct bodies.

Proposition 2.1 (Body injectivity)**.** *β is injective on the semantic field tuple: if $\beta(\mathsf{fr}) = \beta(\mathsf{fr}')$ then fr and fr' agree on every field of Definition 2.1.*

Proof. Each field occupies a disjoint, fixed byte range (Table 2.1), and each integer field's little-endian encoding is a bijection on its width. A byte-equal body forces range-wise equality, hence field-wise equality. The `pad` bytes are absent from the body, so they do not participate; two frames differing only in padding have equal bodies, which is correct — padding carries no consequence. □

2.3 Committing the payload: the object-reference repurposing

The single subtle move in the encoding is how the frame commits the artifact it is a receipt for. The frame has no dedicated "payload" field; instead it carries eight packed object-reference words `obj_refs[0:8]`, each a `u32`, normally OCEL object handles. The praxis frame-builder repurposes all eight words to hold the 256-bit BLAKE3 digest of the canonical payload bytes, *verbatim*.

Construction 2.1 (Payload commitment)**.** Let $p \in \{0,1\}^*$ be the canonical JSON bytes of an admitted payload and $\mathrm{dg}(p) = \mathsf{H}(p) \in \{0,1\}^{256}$ its digest. Partition $\mathrm{dg}(p)$ into eight 32-bit little-endian words $w_0, \ldots, w_7$ and set $\texttt{obj\_refs}[i] = \texttt{PackedObjRef}(w_i)$. Crucially the raw tuple constructor `PackedObjRef(w)` is used, *not* `PackedObjRef::new`, which would pack a type index into the high 8 bits and truncate the identifier to 24 bits. The raw constructor preserves all 32 bits, so the full 256-bit digest survives into bytes $[24, 56)$ of $\beta(\mathsf{fr})$ intact.

Proposition 2.2 (The frame commits the payload)**.** *Under Construction 2.1, $\beta(\mathsf{fr})$ determines $\mathrm{dg}(p)$, and by Proposition 2.1 any change to $\mathrm{dg}(p)$ changes $\beta(\mathsf{fr})$. Consequently a frame commits not to an opaque object handle but to the content of the artifact, exactly as the keystone's law-bearing frame requires the digest $\mathrm{dg}(a)$ of the artifact to appear inside the frame.*

Proof. Bytes $[24, 56)$ of the body are the eight words $w_0 \dots w_7 = \mathrm{dg}(p)$ by construction; the map $\mathrm{dg}(p) \mapsto (w_0, \dots, w_7)$ is the little-endian bijection $\{0,1\}^{256} \to (\{0,1\}^{32})^8$. Injectivity of β (Proposition 2.1) then gives that distinct payload digests induce distinct bodies. □

Remark (Why not add a field). Repurposing `obj_refs` rather than widening the struct keeps the frame at exactly two cache lines and requires no change to the downstream hashing code: `to_hash_bytes` already serializes all eight words verbatim. This is a recurring discipline in the codebase — commit more meaning without growing the committed object — and it is the byte-level instance of the projection principle: the interior (an arbitrarily large payload) is bound by a fixed-width coordinate (its 256-bit digest inside a 99-byte body).

Chapter 3

The Chain: A Rolling Commitment

3.1 Genesis and the fold

A single frame commits one step. A *chain* commits an ordered history by folding each frame's body onto the running commitment of everything before it.

Definition 3.1 (Genesis)**.** The *genesis* chain value is $\mathsf{g} = \mathsf{H}(\mathbf{0}_{32})$, the BLAKE3 digest of 32 zero bytes. A run may additionally bind a *domain seed* $\mathsf{g}_{\mathrm{dom}} = \mathsf{H}(\text{“}\langle\text{project}\rangle\text{-v}\langle\text{version}\rangle\text{-genesis”})$ so that chains from distinct projects or crate versions cannot cross-verify: a receipt minted under one domain seed is not a valid link of another's chain.

Definition 3.2 (Chain rule)**.** For a frame fr with predecessor commitment h_-, the successor commitment is

$$h_+ \;=\; \mathsf{H}\big(h_- \,\|\, \beta(\mathsf{fr})\big), \tag{3.1}$$

where $\|$ is byte concatenation. A *chain* (ledger) $C = (\mathsf{fr}_1, \ldots, \mathsf{fr}_n)$ has commitments $h_0 = \mathsf{g}$ and $h_t = \mathsf{H}(h_{t-1} \,\|\, \beta(\mathsf{fr}_t))$ for $1 \le t \le n$; its terminal value is h_n.

Remark (The predecessor is mixed twice)*.* The praxis realization mixes h_- into h_+ at two points: once as the frame's own `prior_hash` field (bytes $[67, 99)$ of the body) and again as the seed of the fold in (3.1). This double-mixing predates the current construction and is retained for compatibility. It neither strengthens nor weakens the security argument below — h_+ remains a deterministic function of h_- and fr — so all theorems hold verbatim whether the predecessor is mixed once or twice. We note it only so a reader diffing the code against (3.1) is not surprised.

3.2 Determinism and the single construction site

Proposition 3.1 (The chain is a deterministic fold)**.** *h_n is a deterministic function of $(\mathsf{g}, \mathsf{fr}_1, \ldots, \mathsf{fr}_n)$, computable by a single left fold: $h_n = \big(\lambda h.\, \lambda \mathsf{fr}.\, \mathsf{H}(h \,\|\, \beta(\mathsf{fr}))\big)^{\star}(\mathsf{g},\ (\mathsf{fr}_t)_t)$. Equal inputs give bit-identical h_n.*

Proof. Immediate from (3.1): each step is a pure function of two arguments, and composition of pure functions is pure. BLAKE3 is a fixed function, so no nondeterminism enters. □

Proposition 3.1 has an operational corollary that the codebase enforces structurally. Emission (minting a receipt at manufacture time) and replay (recomputing it later from stored fields) must agree, or tamper detection would raise false positives. The crate guarantees agreement by routing both through one function.

Proposition 3.2 (No emission/replay drift)**.** *Let a receipt be emitted by* `LawObject::receipt_with_record` *and later recomputed by* `ReceiptRecord::recompute_chain_hash`*. Both call the same* `build_admission_frame` *and* `chain_from_frame`*. Hence for identical stored fields they compute identical h_+; any disagreement between a stored* `chain_hash_hex` *and its recomputation is therefore attributable to a change in a stored field, never to divergent code paths.*

Proof. There is a single construction site: `build_admission_frame` is the only function that assembles an `OcelCausalFrame` for a receipt, and `chain_from_frame` the only one that folds it. Both callers invoke these with the record's own fields (`payload_hash`, `prev_chain_hash`, `ReceiptMeta`, `ts_ns`). Function purity (Proposition 3.1) then forces equal outputs on equal inputs, so a recompute mismatch isolates a field mutation rather than a logic difference. □

Proposition 3.3 (Prefix commitment)**.** *For each t, h_t commits every frame $\mathsf{fr}_1, \ldots, \mathsf{fr}_t$ and the genesis value: h_t is a function of all of them, and (under the collision-resistance axiom of the next chapter) any change to any of them changes h_t except with negligible probability. In particular the terminal h_n commits the entire history.*

Proof. By induction on t. Base: $h_0 = \mathsf{g}$. Step: $h_t = \mathsf{H}(h_{t-1} \parallel \beta(\mathsf{fr}_t))$ depends on h_{t-1} (which by hypothesis commits $\mathsf{fr}_1, \ldots, \mathsf{fr}_{t-1}$ and g) and on $\beta(\mathsf{fr}_t)$. So h_t depends on all of $\mathsf{g}, \mathsf{fr}_1, \ldots, \mathsf{fr}_t$. The collision clause is deferred to Theorem 4.1. □

3.3 Git-as-a-Runtime CAS Locking and Ledgers

To prevent coordination drift in local-first, air-gapped environment execution, we rely on the host repository's reference structures to enforce atomic locks.

Definition 3.3 (Git CAS Lock)**.** An atomic Compare-And-Swap (CAS) lock for resource L at repository HEAD OID H is acquired by invoking the reference update:

$$\texttt{git update-ref refs/locks/}L\ H\ \mathbf{0}, \tag{3.2}$$

which fails with a non-zero exit code if `refs/locks/`L already exists, and succeeds atomically otherwise via POSIX rename. Deletion on drop releases the lock.

Proposition 3.4 (Append-Only notes Ledger)**.** *The audit ledger is stored under the custom namespace* `refs/notes/praxis/audit`*. Appending entries is a commit transition in which the hash-chain receipts are written as NDJSON notes, preserving the immutability of the execution timeline.*

Proof. Let the audit ledger be represented as a sequence of NDJSON receipt objects. Under the Git Merkle DAG model, each note update is represented as a commit object pointing to a Git tree object that associates target object hashes (OIDs) with note payload blobs. A modification of any existing audit receipt blob at OID H_k changes its content, which under the cryptographic collision-resistance property of SHA-1/SHA-256 (Axiom 4.1) propagates up the Merkle DAG,

resulting in a distinct commit hash at the notes reference `refs/notes/praxis/audit`. Concurrently, the acquisition of a Git CAS lock (Definition 3.3) ensures that only one writer can atomically perform the reference transition: `git update-ref refs/locks/`L H **0** prevents race conditions, ensuring that concurrent writers cannot silently overwrite or split histories. Since the parent commit reference of each note update is cryptographically tied to the prior state, any attempt to insert, delete, or modify historical records requires finding a hash collision or breaking the CAS lock concurrency guarantee. Thus, under the assumption of hash collision resistance, the execution timeline remains cryptographically immutable and append-only. □

Chapter 4

The Faithful Chain Theorem

We now discharge the keystone's standing assumption by stating it as an explicit axiom and proving what it buys.

Axiom 4.1 (Collision resistance)**.** H (instantiated as BLAKE3 [2]) is collision-resistant: no probabilistic polynomial-time adversary finds $x \neq y$ with $\mathsf{H}(x) = \mathsf{H}(y)$ except with negligible probability $\varepsilon(\lambda)$ in the security parameter λ. For BLAKE3, $\lambda = 256$ and the best generic attack is the birthday bound $\sim 2^{128}$.

Theorem 4.1 (Faithful Chain)**.** *Let $C = (\mathsf{fr}_1, \ldots, \mathsf{fr}_n)$ be a chain with terminal commitment h_n, and let $C' = (\mathsf{fr}'_1, \ldots, \mathsf{fr}'_n)$ differ from C in at least one committed field of at least one frame (any semantic field of Table 2.1, including the payload digest of Construction 2.1). If C' has the* same *terminal commitment h_n, then a collision of H has been exhibited. Consequently, under Axiom 4.1, an adversary who alters any committed field yet preserves h_n succeeds only with probability $\leq \varepsilon(\lambda)$.*

Proof. Let j be the least index at which C and C' differ in a committed field. Bodies are injective on committed fields (Propositions 2.1, 2.2), so $\beta(\mathsf{fr}_j) \neq \beta(\mathsf{fr}'_j)$. Because j is least, the predecessors agree: $h_{j-1} = h'_{j-1}$. Consider the two fold inputs $x = h_{j-1} \parallel \beta(\mathsf{fr}_j)$ and $x' = h'_{j-1} \parallel \beta(\mathsf{fr}'_j)$. Since the prefixes $h_{j-1} = h'_{j-1}$ are equal and have equal length (32 bytes) while the suffixes differ, $x \neq x'$. Now examine the terminal values. If $h_j \neq h'_j$, then by Proposition 3.3 the divergence propagates: each subsequent fold takes a differing 32-byte prefix, so equality of the terminal $h_n = h'_n$ would require some fold step $t \geq j$ to map unequal inputs to equal outputs — a collision. If instead $h_j = h'_j$ despite $x \neq x'$, then $\mathsf{H}(x) = \mathsf{H}(x')$ is itself a collision at step j. In either case $h_n = h'_n$ forces a collision, contradicting Axiom 4.1 up to $\varepsilon(\lambda)$. □

Corollary 4.2 (Bounded verification cost)**.** *A verifier checking a single frame recomputes one fold* (3.1) *— one BLAKE3 of a $32 + 99 = 131$-byte input — and one equality test: $O(1)$ time and space, independent of n and of the interior dimension of the manufacture the frame receipts. Checking the boundary projection of a whole run (verdict, terminal h_n, fitness scalar, refusal reason) is $O(\kappa)$, matching the keystone's Comprehension–Verification Gap.*

Proof. The fold reads two fixed-width operands and produces a 256-bit output; BLAKE3 on a fixed-length input is constant work. Equality on 256-bit values is constant. The boundary projection has $\leq \kappa$ chunks by construction (keystone, Def. receipt projection), so applying the boundary predicate is $O(\kappa)$. Neither cost references n or $\dim \Sigma$. □

Remark (Faithful *only* over committed fields). Theorem 4.1 is deliberately scoped to committed fields. A change confined to a field *absent* from β — the `pad` bytes, or the descriptive `activity` label and `duration_ms` of `ReceiptRecord`, which are marked non-hashed — does not change h_n and is, correctly, *not* attested. Faithfulness is a property of the commitment, and the commitment is exactly Table 2.1. This is the byte-exact instance of the keystone's converse clause: a frame committing bytes alone would attest tamper-evidence but not lawfulness; a frame committing the denial word, the payload digest, and the transition identity attests those. What you commit is what you can trust; no more, and no less.

Chapter 5

Conservation of Consequence

The keystone stated Conservation of Consequence abstractly. Here it becomes a theorem about chain positions, provable directly from Definitions 3.2 and the Chatman equation.

Theorem 5.1 (Conservation of Consequence, receipt form)**.** *Fix the Chatman equation* $\mathcal{A} = \mu(\mathcal{O}^*)$ *with the receipt chain of Definition 3.2. Then:*

(i) ***Caused.*** *Every actuated artifact* a *satisfies* $a = \mu(x)$ *for some admitted* $x = \alpha(o) \neq \bot$*; no artifact outside* $\operatorname{im}\mu$ *is actuated, and a receipt is minted only for an object that reached the* `Admitted` *stage.*

(ii) ***Positioned.*** *Each actuation appends* exactly one *frame and advances the chain by* exactly one *commitment; the map* $\Psi : \textit{actuation} \mapsto h_+$ *is injective up to hash collision.*

(iii) ***Attributed.*** *The committed frame carries at least one admitted cause: bytes* $[24, 56)$ *commit* $\mathrm{dg}(x)$ *(the admitted-payload digest, Construction 2.1) and bytes* $[16, 24)$ *commit the denial word certifying* x *passed admission.*

Proof. (i) Actuation is gated by the equation: an artifact enters $\mathcal{A}$ only as $\mu(x)$ with $x \neq \bot$, so $\operatorname{im}\mu$ exhausts the actuated set. In the code, `receipt` is a method available only on `LawObject<_, Admitted, _>`; the typestate makes "receipt without admission" uninhabited (foundations paper), so no artifact is receipted without an admitted antecedent.

(ii) Each call to `chain` increments `frame_count` by one and produces one new `chain_hash`; (3.1) appends exactly one commitment per frame. Suppose two distinct actuations mapped to the same h_+. Their frames differ (distinct instruction identity in `instruction_id`, or distinct payload digest), so by Theorem 4.1 equal terminal commitments force a collision. Hence Ψ is injective up to $\varepsilon(\lambda)$.

(iii) By Construction 2.1, $\mathrm{dg}(x)$ occupies bytes $[24, 56)$; the denial word `denial.0` occupies $[16, 24)$ and equals `ADMITTED` on the accept path (the record persists an admitted-polarity frame). Thus the frame commits both the identity of the cause and the verdict that admitted it. □

Corollary 5.2 (No orphan and no double-spend of consequence)**.** *No actuated artifact lacks a chain position (*no orphan*: by (i)+(ii)), and no chain position is shared by two actuations (*no double-count*: by (ii)). Consequence is neither created without an admitted cause nor recorded without a unique receipt slot.*

Proof. Let A be the set of actuated artifacts and C be the set of chain positions represented by commitments h_i. By Theorem 5.1 (i), every actuated artifact $a \in A$ has a corresponding admitted cause $x = \alpha(o) \neq \bot$ that triggers an actuation. By Theorem 5.1 (ii), each such actuation maps to a new unique commitment $h_+ \in C$ under the commitment map Ψ, which is injective up to hash collision. First, if there exists an actuated artifact a that lacks a chain position (an orphan), this directly contradicts Theorem 5.1 (ii) which states that each actuation appends exactly one frame and advances the chain by exactly one commitment, establishing a valid position for all actuated consequences. Second, if a chain position is shared by two distinct actuations (double-spend of consequence), then we have two distinct actuations mapping to the same commitment h_+. However, because the commitment map Ψ is injective up to hash collision (Theorem 5.1 (ii)), this equality of commitments implies a hash collision has been found. Under Axiom 4.1, such a collision occurs only with negligible probability $\varepsilon(\lambda)$. Therefore, consequence is neither created without an admitted cause nor recorded without a unique, non-overlapping receipt slot. □

Remark (Uniqueness of cause is by commitment, not by inversion). As the keystone stressed, μ need not be injective: two admitted observations may manufacture the same artifact. Clause (iii) does not claim a determines x; it claims the frame *commits* $\mathrm{dg}(x)$, so h_+ pins the cause even where the artifact alone would not recover it. Uniqueness of the cause is a property of the receipt, not a false property of the morphism.

Chapter 6

Tamper Localization: The Staged Validator

Theorem 4.1 gives a *detector*: a tampered ledger fails to verify. A production verifier must do more — it must say *where* and *how* the ledger is broken, because at fleet scale "this ledger is invalid" is an alarm with no address. The praxis `ReceiptValidator` is a staged pipeline that localizes the fault.

Definition 6.1 (Validation pipeline)**.** The validator runs five stages in fixed order over a ledger $(\mathsf{fr}_t)_{t=1}^{n}$, each returning `Pass`, `Fail`(message), or `Skip`(reason), and reports *all* stage outcomes rather than short-circuiting:

(S1) `schema` — every record has the supported version and well-formed 32-byte hex fields (`payload_hash`, `prev_chain_hash`, `chain_hash`).

(S2) `chain_recompute` — for each t, `recompute_chain_hash` equals the stored `chain_hash_hex`. This is the direct instantiation of Theorem 4.1: a mismatch is a committed-field tamper.

(S3) `chain_linkage` — for each $t > 1$, `prev_chain_hash_hex`(t) = `chain_hash_hex`$(t-1)$: the records are actually threaded, not merely individually valid.

(S4) `monotonic` — `instruction_id` is strictly increasing, `ts_ns` is non-decreasing, and no `ts_ns` exceeds the (injectable) clock's "now".

(S5) `token_replay` — each record replays through the fixed POWL token model to fitness exactly `0x0001_0000` (§6.1).

Theorem 6.1 (Tamper localization)**.** *Suppose a lawful ledger C is perturbed to C' by a modification whose first affected record is index j. Then the pipeline of Definition 6.1 reports a* `Fail` *whose named record index is j (equivalently, the least index whose recomputation, linkage, monotonicity, or replay breaks), and the stage identifier names the class of the fault. The honest prefix $\mathsf{fr}_1, \ldots, \mathsf{fr}_{j-1}$ is not implicated.*

Proof. The stages scan records in increasing index and each returns on its *first* failing record. Consider the classes of a single-record perturbation at j:

- If the mutation malforms a hex field, `schema` fails at j (earlier records parse, being unperturbed).

- If the mutation changes a committed field but leaves `chain_hash_hex`(j) stale, `chain_recompute` fails at j: by Proposition 3.2 the recomputation uses the perturbed fields, so it diverges from the stored value exactly at j; records $< j$ are unperturbed and recompute equal.
- If the mutation reorders or splices records so that thread continuity breaks (e.g. a swap), `chain_linkage` fails at the least index whose `prev` no longer matches its predecessor's `chain_hash`.
- If the mutation rewrites both a field *and* its stored chain hash to be internally consistent (a coordinated forgery), then either linkage breaks at j (the forged `chain_hash`(j) no longer equals what record j+1 expects, unless the adversary rewrites the entire suffix), or, if the whole suffix is rewritten, the forged terminal commitment differs from the attested h_n and Theorem 4.1 applies at the boundary — an anchored h_n (e.g. a signed or externally published terminal value, see Chapter 7) makes suffix rewriting a collision problem.
- If the mutation preserves hashes but produces a trace that is not a firing sequence, `token_replay` fails at j with fitness < 1 (§6.1).

In every class the least failing index is j and the honest prefix passes every stage, because each stage's predicate on records $< j$ is evaluated on unperturbed data and holds by hypothesis. □

Remark (Report-all, not fail-fast, across stages). Within a stage the scan returns on the first offending record (so the *record* is localized); across stages the pipeline runs every stage and reports each outcome (so the *fault class* is fully characterized, and a ledger broken in two independent ways shows both). This mirrors the affidavit-style `verify` pipeline's "run every stage, report all of them" shape and is what lets a downstream consumer act on the specific defect rather than a bare Boolean.

6.1 Fitness as the replay coordinate

Definition 6.2 (Replay fitness)**.** Replay fitness is the normalized conformance metric of the keystone's marking geometry, $\varphi = 1 - \frac{\text{tokens forced on unenabled transitions}}{\text{tokens the replay attempted}} \in [0, 1]$, represented in the code as a Q16.16 fixed-point integer: the value 1.0 is the constant `0x0001_0000` = 65536. The `token_replay` stage accepts a record iff its replayed fitness equals `0x0001_0000`.

Proposition 6.2 (Fitness = 1 characterizes lawful replay)**.** *A record's replay attains $\varphi =$ `0x0001_0000` iff its lifecycle is a genuine firing sequence of the fixed POWL token model; otherwise the first forced consumption localizes the nonconformant step, and the stage fails naming that record.*

Proof. By the keystone's unit-fitness characterization: enabled firings force no consumption, giving numerator 0 and $\varphi = 1$ (= `0x0001_0000` in Q16.16); a disabled event forces a consumption, making the numerator positive and $\varphi < 1$, at the earliest offending index. The stage compares against the exact fixed-point unit. □

Chapter 7

Authenticity: Ed25519 as a Self-Contained Signature

The chain answers (Q1), integrity: *was this history altered?* It does not by itself answer (Q2), authenticity: *who vouches for it?* An adversary who fabricates an entirely fresh but internally consistent ledger produces a valid chain — valid chains are cheap to manufacture; what is expensive is manufacturing one that collides with a *committed* prior value. Authenticity is added by binding the terminal commitment to a keyholder.

Definition 7.1 (Signed receipt)**.** A *signed receipt* is the triple $\texttt{SignedReceipt} = \langle h_{\text{hex}},\ s,\ k \rangle$ where h_{hex} is the hex terminal chain value, $s = \text{Sign}_{sk}(h_{\text{hex}})$ is an Ed25519 signature [3] over it, and k is the hex verifying key. It is *self-contained*: s and k travel with h_{hex}, so the object verifies without out-of-band key material.

Proposition 7.1 (Integrity vs. authenticity are distinct checks)**.** *There are two verification predicates:*

- ***Integrity*** *(`verify_chain_hash`): check s against the* embedded *key k and that k's signature covers the presented h. This proves the signed object was not altered after signing, but trusts whatever key is inside it.*
- ***Authenticity*** *(`verify_chain_hash_with_key`): check s against an* externally pinned *key $k^\star$. This fails if the receipt was signed by any key other than $k^\star$, even if internally consistent.*

The first is necessary but not sufficient for the second; a verifier who wants "this came from the party I trust" must use the second and supply $k^\star$ from an independent source.

Proof. Ed25519 verification is a function of $(\text{message}, \text{signature}, \text{key})$. `verify_chain_hash` instantiates the key argument with the embedded k; any adversary who re-signs a modified h with a self-generated key k' passes this check, so it does not establish authorship. `verify_chain_hash_with_key` instantiates the key argument with $k^\star$; by Ed25519's existential unforgeability [3], producing a valid s over h under $k^\star$ without $sk^\star$ is infeasible, so a pass attributes authorship to the holder of $sk^\star$. The embedded-key check therefore cannot substitute for the pinned-key check. □

Proposition 7.2 (Fail-closed signing)**.** *When the `signed` feature is enabled, a missing or unreadable signing key (`PRAXIS_SIGNING_KEY` / `_FILE`) is a hard error, not a silent skip: the signing path returns `SigningFailed` rather than emitting an unsigned receipt.*

Proof. The `signed` feature exists to *guarantee* a signature; silently skipping would defeat its purpose and let an unsigned receipt masquerade as covered. The implementation therefore treats key absence as failure. Authenticity is thus either present and checkable or explicitly absent, never ambiguously assumed. □

Remark (What a signature adds to the chain, precisely). Composing Theorem 4.1 with Proposition 7.1: the chain makes *internal* tampering a collision problem; the signature makes *substituting a different history* an unforgeability problem, provided the verifier pins $k^\star$ and treats h_n as the anchored value. Neither alone suffices against a resourceful adversary; together they reduce forgery to breaking BLAKE3 collision resistance *or* Ed25519 unforgeability.

Chapter 8

The Receipt Lake: Content-Addressed and Queryable at Index Scale

Individual receipts chain into a per-run ledger. Across a fleet, ledgers accumulate into a *receipt lake* L: a content-addressed, append-only store engineered so that the population of receipts is queryable without reading interiors.

Definition 8.1 (Content addressing)**.** The address of a stored object b is $\texttt{content\_address}(b) = \mathsf{H}(b)$ in hex. Two byte-identical objects share an address; any change yields a different address. A receipt's own terminal h_n is such an address, so a receipt is stored under a name that is a function of its content.

Proposition 8.1 (De-duplication and immutability are free)**.** *In a content-addressed lake, (a) storing the same receipt twice is idempotent (same address), and (b) mutating a stored receipt changes its address, so the old address still resolves to the old bytes: history is immutable by construction, and references are stable.*

Proof. (a) The address is $\mathsf{H}(b)$, a function of b; equal b gives equal address. (b) A change $b \to b'$ with $b \neq b'$ gives $\mathsf{H}(b') \neq \mathsf{H}(b)$ except on a collision (Axiom 4.1); the reference $\mathsf{H}(b)$ therefore cannot be made to resolve to b'. □

8.1 OCEL 2.0 and PROV-O mappings

A receipt is not only a hash; it carries process structure. Each `ReceiptRecord` names the OCEL objects it governs via `object_ids` (event-to-object links), an `activity_idx` and `node_kind` placing it in a POWL process, and an `instruction_id` sequencing it. This lets the ledger export as a standard event log.

Construction 8.1 (OCEL 2.0 export)**.** The map $\Omega : C \to \text{OCEL2.0}$ sends each frame to an OCEL *event* with: event id = `instruction_id`; activity = label of `activity_idx`; timestamp = `ts_ns`; and event-to-object (E2O) qualifiers to each identifier in `object_ids`. The result is a conformant OCEL 2.0 log queryable by any process-mining tool, produced by `receipt export-ocel`.

Construction 8.2 (PROV-O / SHACL bridge)**.** Each receipt maps to a PROV-O [4] shape: the admitted observation is a `prov:Entity`, the manufacture a `prov:Activity` that `prov:used` it, the artifact a `prov:Entity` that `prov:wasGeneratedBy` the activity, and the chain link a `prov:wasDerivedFrom` edge to the predecessor. The praxis `receipt_shacl` module realizes a concrete instance, validating a `ReceiptRecord` against the `sr:SharedReceiptV1` SHACL shape — so a receipt is not merely serializable to RDF but *shape-checked* against a published contract before it enters the lake.

Proposition 8.2 (The mappings preserve the commitment)**.** Ω *and the PROV-O bridge are lossless over the committed fields: the terminal* h_n *is recoverable from the exported log's per-event fields via Definition 3.2, so exporting to OCEL 2.0 or RDF does not weaken the chain guarantee — a consumer can recompute and re-verify from the exported form.*

Proof. Ω carries each frame's committed fields (`instruction_id`, `activity_idx`, `ts_ns`, the object identifiers, and the hex hashes) into event attributes; the chain recomputation of Proposition 3.2 depends only on these plus `payload_hash` and `prev_chain_hash`, which the record retains. Hence the exported log determines h_n, and re-verification is Theorem 4.1 applied to the reconstructed frames. □

8.2 Index-scale query: the QLever direction

Remark (Querying a lake without reading interiors)*.* Once receipts are RDF triples (Construction 8.2), the population is a knowledge graph. An engineered SPARQL index such as QLever [6] answers queries over 10^{11}-scale triple stores by a join plan no engineer reconstructs — the very "differential agreement over a checked projection" the keystone cites as manufactured trust. The design direction is: the lake stores content-addressed receipts; a triple index over their committed fields answers fleet-scale questions ("which agents refused with category c in window w?", "what is the pass-rate over frontier F?") at index speed, touching only the $O(\kappa)$-sized committed coordinates, never the interiors. This is the projection principle at the scale of a whole civilization's audit log: the interior is a trillion manufactures; the queryable surface is their committed frames.

8.3 Git-CAS Locking and Crash Recovery

The `chatman-common` crate realizes atomic receipt ledger writes by mapping the receipt lock problem onto the host Git object store. For an air-gapped, local-first execution environment where no external coordinator is available, this is the correct level of mechanism: Git's reference-update protocol uses POSIX rename, which is atomic on POSIX-compliant filesystems.

Definition 8.2 (Git CAS Lock, operational form)**.** Let L be a named resource (e.g. the praxis audit ledger) and let H be the current HEAD OID of the local repository. The CAS lock for L is acquired by

$$\texttt{git update-ref refs/locks/}L\ H\ \mathbf{0}_{20},$$

which atomically creates `refs/locks/`L pointing to H, succeeding iff the reference does not already exist (the old-value argument $\mathbf{0}_{20}$ means "require absent"). Deletion of the reference on drop releases the lock. The lock is therefore held for at most the duration of one `git`

`update-ref` round-trip plus the critical section, without holding any OS-level file lock that could block unrelated operations.

Proposition 8.3 (Append-only notes ledger under CAS)**.** *The audit ledger is stored under the custom Git notes namespace* `refs/notes/praxis/audit`*. Under the CAS lock of Definition 8.2, each append of a receipt record is a* commit transition*: the new record is serialized as NDJSON and appended to the notes tree under the commit OID it annotates. Because Git commits are content-addressed (the commit hash is the BLAKE3 of its tree plus metadata), the appended entry is immutable by Git's object model. The notes reference advances atomically under the CAS guard, so two concurrent append attempts on the same ledger are serialized by the acquire-fail branch of the CAS.*

Proof. A failing CAS (reference already exists) exits with a non-zero code before touching the notes tree; only the lock holder proceeds. Within the critical section, the notes append is a single `git notes add` followed by the reference-delete on the lock. If the process is killed after the notes append but before the lock release, the lock entry remains; the restart protocol in `chatman-common` detects a stale lock by comparing the locked OID H to the current HEAD — if they differ (a new commit has been pushed), the lock is stale and is force-released, because the locked OID no longer uniquely identifies a live critical section. □

Remark (Relationship to the Faithful Chain Theorem)*.* The CAS lock protocol is orthogonal to the hash chain: the chain guards *what was recorded* (no field tampered), while the CAS lock guards *that only one writer appended at a time* (no race). Together they give a ledger that is simultaneously append-sequential (by the lock) and tamper-evident (by the chain). Violation of sequentiality (two writers appending concurrently) would not break Theorem 4.1; it would produce a fork in the notes tree rather than a hash collision. The CAS lock prevents the fork; the chain prevents the tamper; neither subsumes the other.

Chapter 9

Integrity Is Not Virtue: The Scope Theorem

The most important theorem in a cryptographic doctrine is the one that says what the cryptography does *not* prove. Without it, a receipt chain becomes a laundering device: tamper-evidence dressed up as a warrant of goodness. We state the boundary precisely.

Theorem 9.1 (Integrity–Virtue Scope)**.** *Let a receipt chain C verify (Theorem 4.1) and, optionally, be authentically signed (Proposition 7.1). Then C establishes exactly the following, and no more:*

(A) ***Integrity.*** *The committed fields of every frame are as they were when minted; no committed field was altered post hoc (up to $\varepsilon(\lambda)$).*

(B) ***Attribution.*** *Each artifact is committed to an admitted cause and a denial word, and (if signed) to a keyholder.*

(C) ***Conformance-as-checked.*** *Each replayed lifecycle is a firing sequence of the* fixed *POWL model to fitness* `0x0001_0000` *(Proposition 6.2).*

It does ***not*** *establish:*

(a) ***Obligation adequacy.*** *That the obligation battery G was the* right *set of checks — the chain commits* $\mathrm{dg}(G)$ *and the denial word, not the wisdom of choosing G.*

(b) ***World-fitness.*** *That the artifact is correct, safe, useful, or ethical in the world — μ is deterministic and bounded, not benevolent.*

(c) ***Semantic truth.*** *That the admitted observation* meant *what it purported; admission retracted onto a decidable sub-language (Rice quarantine), it did not decide meaning.*

Proof. (A) is Theorem 4.1: altering a committed field forces a collision. (B) is Conservation of Consequence (iii) (Theorem 5.1) composed with Proposition 7.1. (C) is Proposition 6.2, and is scoped to the *fixed* model: fitness certifies conformance to the model that was replayed, not that the model is the correct model.

For the negatives, we exhibit that each is independent of the verifying predicate. (a) Two ledgers with different obligation sets $G \neq G'$ can both verify; the chain commits $\mathrm{dg}(G)$ so it

records which set was used, but Verify returns `Pass` for any committed G, so verification does not rank G against G'. (b) Fix any admitted x; both a benign and a harmful deterministic μ produce verifying chains, since verification never inspects the world-consequences of $a = \mu(x)$ — only that a's digest is committed. Hence world-fitness is not a function of the verdict. (c) By the series' Rice specialization, no computable predicate decides the semantic property "o means what it purports"; admission is a retraction, not a decision, so a verifying receipt attests admission, which is provably weaker than truth. Each negative is therefore not entailed by (A)–(C). □

Corollary 9.2 (The antibody clause, cryptographic form)**.** *A claim of the form "this artifact is* good *because its receipt verifies" is an overclaim: it asserts a virtue (b) that the verifying predicate provably does not entail (Theorem 9.1). The admissible claim is narrower and exact: "this artifact was manufactured from an admitted cause under obligation set G, unaltered and (optionally) signed by $k^\star$, conformant to model M." A discipline that only emits the narrow claim is machinery; one that emits the broad claim is grandiosity.*

Proof. The broad claim entails (b), which Theorem 9.1 shows is independent of the verdict; so the broad claim is not supported by the receipt. The narrow claim is exactly the conjunction (A)∧(B)∧(C), each proven. Emitting only what is entailed is the keystone's `docs-exceed-mechanism` invariant applied to cryptographic claims. □

Chapter 10

Conclusion

We built the cryptography the keystone assumed. A frame is a fixed-width, 128-byte, cache-aligned encoding of one manufacturing step whose 99-byte hash body commits the full payload digest, the denial word, and the transition identity (Chapter 2). A chain folds frames by $h_+ = \mathsf{H}(h_- \parallel \beta(\mathsf{fr}))$ into a rolling commitment computed by a single, drift-free construction site (Chapter 3). From an explicit collision-resistance axiom we proved the Faithful Chain Theorem — perturbing any committed field forces a collision, and a bounded verifier rejects it reading $O(1)$ per frame (Chapter 4) — and Conservation of Consequence in receipt form: exactly one chain position per actuation, injective up to collision, each artifact committed to an admitted cause (Chapter 5). The staged validator does not merely detect tampering but localizes it to the first divergent frame and names the fault class (Chapter 6). Ed25519 signatures add self-contained authenticity, kept distinct from integrity and made fail-closed (Chapter 7). Receipts accumulate into a content-addressed lake with lossless OCEL 2.0 / PROV-O mappings and an index-scale query direction (Chapter 8).

The final theorem is the one a cryptographer must never omit: integrity is not virtue (Chapter 9). A verifying, signed chain proves that a history is unaltered, attributed, and conformant-as-checked — and proves, with equal rigor, that it says nothing about whether the obligations were wise, the artifact good, or the observation true. That boundary is not a weakness of the receipt. It is the discipline that lets a receipt be trusted for exactly what it commits, at a scale where trusting it for more would be the first lie a trillion agents tell each other.

A receipt commits; it does not absolve.

Appendix A

Notation and Field Layout

Symbol	Meaning
fr, $\beta(\mathsf{fr})$	frame; its 99-byte canonical hash body (Table 2.1)
H, $\mathrm{dg}(\cdot)$	BLAKE3 hash; digest $\mathrm{dg}(b) = \mathsf{H}(b)$
h_-, h_+, h_t	predecessor / successor / step-t chain commitment
g	genesis commitment $\mathsf{H}(\mathbf{0}_{32})$, optionally domain-bound
$\|$	byte concatenation
C, L	chain (ledger); content-addressed receipt lake
$\varepsilon(\lambda)$	negligible collision/forgery probability; $\lambda = 256$
Ψ	actuation $\mapsto$ terminal commitment map (injective up to collision)
φ	replay fitness; unit = `0x0001_0000` (Q16.16)
G, $\mathrm{dg}(G)$	obligation battery; its committed digest
k, $k^\star$, sk	embedded / pinned verifying key; signing key
Ω	OCEL 2.0 export map (Construction 8.1)

Code sites. `OcelCausalFrame`, `to_hash_bytes`, `OcelCausalReceipt::chain` (`bcinr-powl-receipt`); `build_admission_frame`, `chain_from_frame`, `ReceiptRecord`, `recompute_chain_hash`, `ReceiptValidator`, `signing` (`praxis-core`); `content_address`, `SignedReceipt` (`chatman-common`); `receipt_shacl` (root crate).

Bibliography

[1] The Praxis / Capability Physics Program. *The Chatman Equation and the Industrial Revolution of Knowledge*, v1.2.0, Nov. 2025.

[2] J.-P. Aumasson, S. Neves, Z. Wilcox-O'Hearn, J. O'Connor. *BLAKE3: One Function, Fast Everywhere*. 2020.

[3] D. J. Bernstein, N. Duif, T. Lange, P. Schwabe, B.-Y. Yang. *High-speed high-security signatures* (Ed25519). J. Cryptographic Engineering, 2012.

[4] T. Lebo, S. Sahoo, D. McGuinness (eds.). *PROV-O: The PROV Ontology*. W3C Recommendation, 2013.

[5] A. Berti, I. Koren, J. N. Adams, et al. *OCEL (Object-Centric Event Log) 2.0 Specification*. 2023.

[6] H. Bast, B. Buchhold. *QLever: A Query Engine for Efficient SPARQL+Text Search*. CIKM, 2017.

[7] H. G. Rice. *Classes of recursively enumerable sets and their decision problems*. Trans. AMS, 1953.